

«Национальный исследовательский университет «Московский институт
электронной техники»

Мобильные системы связи.

Лабораторная работа №2

«Модуляция и кодирование в стандарте LTE»

Выполнил студент группы ИКТ-31:

Кулигин Артём

Зеленоград, 2026 г.

Задание №1: Для каждого типа модуляции (QPSK, 16QAM, 64QAM) получить и сравнить BER при жестком и мягком декодировании.

В этом задании нам необходимо провести моделирование для получения графиков BER разных модуляций, используемых в технологии LTE - QPSK, 16-QAM, 64-QAM.

Будем использовать следующие **функции** для реализации нашей задачи:

lab2_ex01.m - функция, в которой используется демодулятор на основе декодирования с жестким решением

lab2_ex02.m - функция, в которой используется демодулятор на основе декодирования с мягким решением

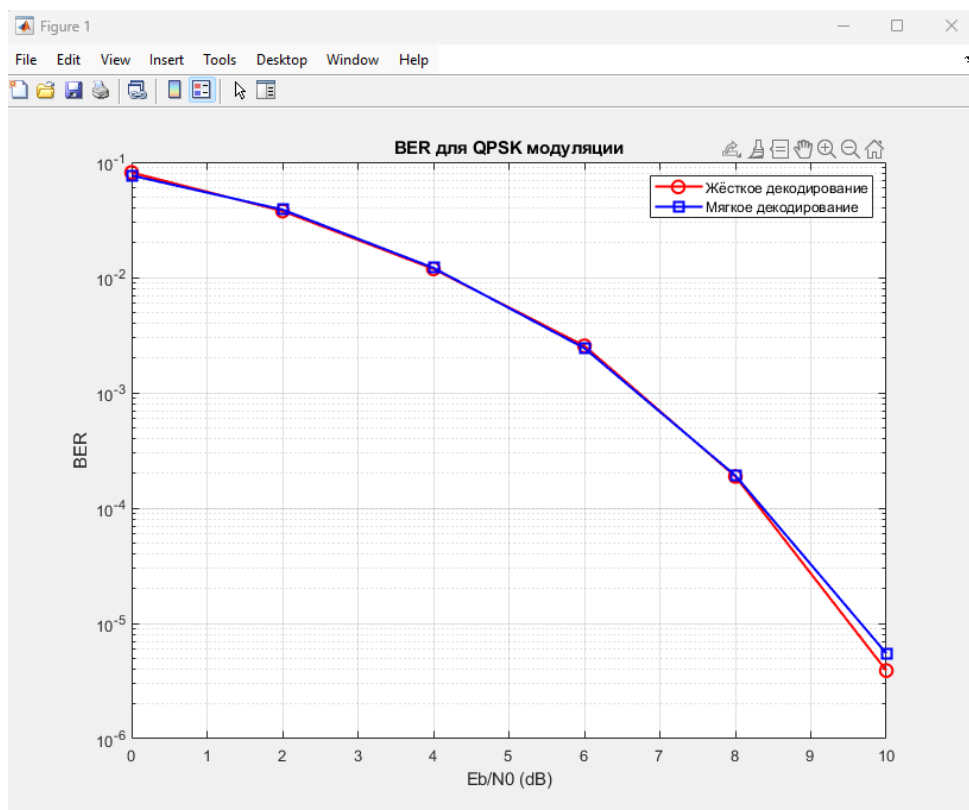


Рисунок 1 - График BER (QPSK) с применением жесткого и мягкого декодирования

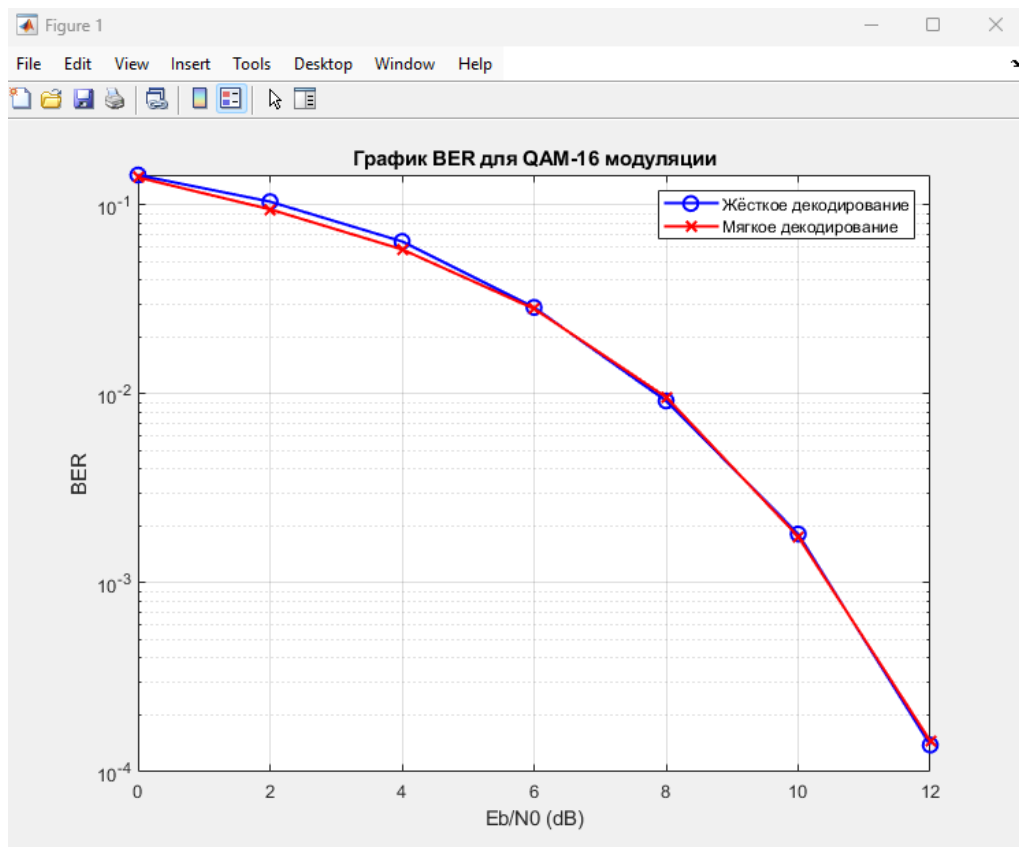


Рисунок 2 - График BER (16-QAM) с применением жесткого и мягкого декодирования

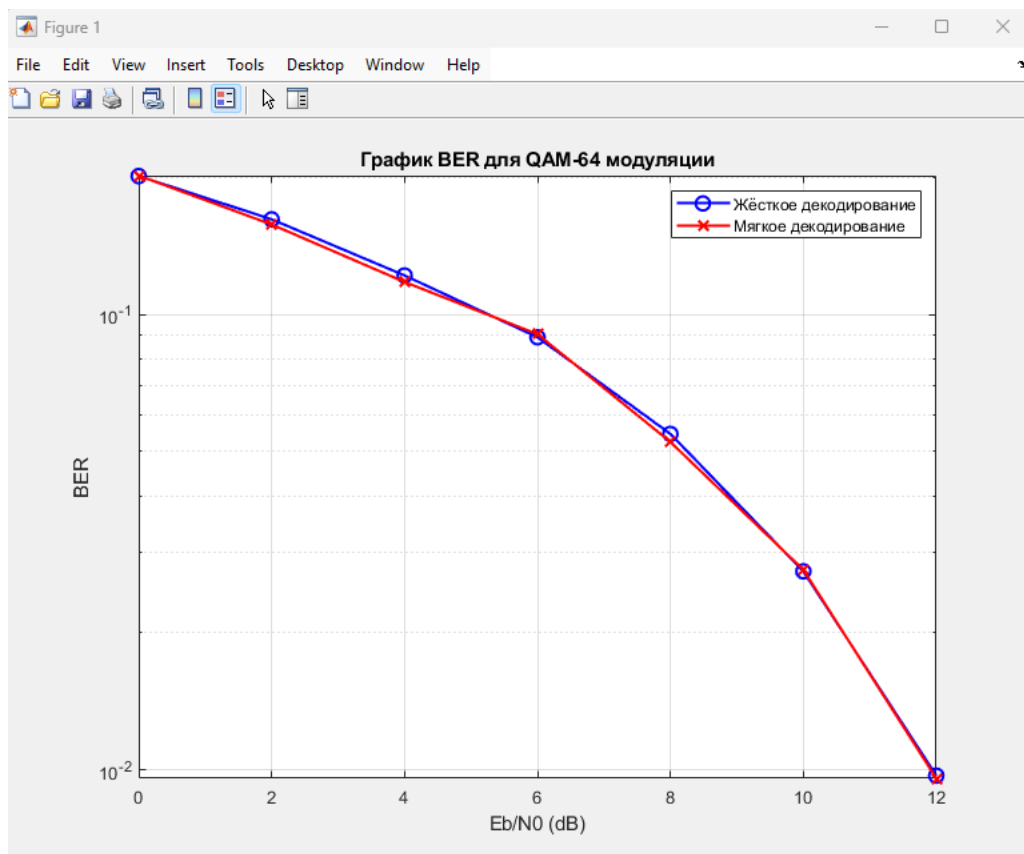


Рисунок 3 - График BER (64-QAM) с применением жесткого и мягкого декодирования

Из полученных графиков мы приходим к следующим выводам: в условиях не кодированной передачи по каналу с АБГШ жесткое и мягкое декодирование показывают ту же самую эффективность, поскольку демодулятор обрабатывает каждый символ независимо и не может извлечь выгоду из информации о надежности, даже если она вычисляется.

Так же стоит упомянуть, что скремблирование, выполняющее операцию «исключающего ИЛИ» с псевдослучайной последовательностью, не вносит избыточности и не изменяет статистику ошибок, поэтому его присутствие никак не сказывается на сравнении двух подходов. Полное совпадение кривых BER на всем диапазоне E_b/N_0 подтверждает, что для не кодированных систем мягкие метрики не дают преимущества.

Задание №2: Определение последовательности Голда. Главные свойства последовательностей. Как их получают. Роль в технологии LTE.

Последовательности Голда — это класс псевдослучайных двоичных последовательностей, обладающих уникальными корреляционными свойствами. Они были предложены Робертом Голдом в 1967 году как компромиссное решение между желанием иметь большое количество последовательностей с хорошими свойствами и ограничениями.

Главные особенности последовательностей Голда:

1. Они формируются из суммы по модулю два (XOR) двух предпочтительных m -последовательностей, что позволяет получать большое количество различных последовательностей при относительно простой реализации.
2. Они обладают строго ограниченной взаимной корреляцией, то есть любые две последовательности из одного набора остаются практически некоррелированными, что дает возможность их одновременного использования в одном частотном диапазоне без взаимных помех.
3. Автокорреляционная функция каждой последовательности имеет ярко выраженный пик при нулевом сдвиге и низкий уровень боковых лепестков, что обеспечивает надежную синхронизацию и обнаружение сигнала даже в условиях сильных шумов в каналах.

Как получают те самые последовательности Голда?

Распишем их получение поэтапно:

1 этап заключается в выборе пары предпочтительных m -последовательностей. Для этого берут два регистра сдвига одинаковой длины,

каждый из которых генерирует m -последовательность. При этом пару называют предпочтительной, если их взаимная корреляционная функция принимает только три возможных значения, что гарантирует оптимальные свойства будущего набора.

2 этап состоит в формировании опорной последовательности. Один из регистров фиксируется в некотором начальном состоянии, а второй регистр также инициализируется определенным образом, после чего оба регистра начинают работать синхронно, сдвигаясь на каждом такте.

3 этап представляет собой непосредственное получение последовательности Голда. На каждом такте выходные биты обоих регистров складываются по модулю два с помощью логической операции исключающего ИЛИ. Полученная на выходе бинарная последовательность и является одной из последовательностей Голда.

Роль последовательностей Голда в технологии LTE:

Формирование опорных сигналов.

Пилотные символы, необходимые для оценки канала и синхронизации, генерируются на основе последовательностей Голда. Благодаря этому каждая сота получает уникальные и легко различимые опорные сигналы.

Генерация сигналов синхронизации.

Первичные и вторичные сигналы синхронизации в LTE строятся с использованием последовательностей Голда, что позволяет абонентскому устройству быстро обнаруживать соту и синхронизироваться с ней по времени и частоте.

Формирование физических каналов управления.

Для каналов, передающих служебную информацию, таких как физический канал управления нисходящей линии связи, последовательности Голда применяются для обеспечения помехоустойчивости и надежного приема даже в сложных условиях.

Разделение пользователей в многоантенных системах.

При использовании технологии MIMO последовательности Голда помогают различать сигналы, передаваемые с разных антенн, что повышает пропускную способность системы.

Таким образом, последовательности Голда в LTE играют роль универсального инструмента для идентификации, разделения и защиты сигналов на всех уровнях передачи данных.

Задание №3: Влияние скремблирования на графики BER для конкретных типов модуляций.

В этом задании было необходимо внести корректировки в функции **lab2_ex01** / **lab2_ex02** для переключения использования скремблирования.

Что мы сделали? Добавили флаг *scramblingFlag*, который отвечает за использование скремблирования, т.е. при значении *true* будет использоваться, а при *false* не будет.

```
EbNo_dB = 0:2:10;
maxErrs = 500;
maxBits = 1e7;

ber_hard_sc = zeros(size(EbNo_dB));
ber_hard_no_sc = zeros(size(EbNo_dB));
ber_soft_sc = zeros(size(EbNo_dB));
ber_soft_no_sc = zeros(size(EbNo_dB));

for i = 1:length(EbNo_dB)
    fprintf('Eb/No = %d dB\n', EbNo_dB(i));
    ber_hard_sc(i) = lab2_ex01(EbNo_dB(i), maxErrs, maxBits, true);
    ber_hard_no_sc(i) = lab2_ex01(EbNo_dB(i), maxErrs, maxBits, false);
    ber_soft_sc(i) = lab2_ex02(EbNo_dB(i), maxErrs, maxBits, true);
    ber_soft_no_sc(i) = lab2_ex02(EbNo_dB(i), maxErrs, maxBits, false);
end

figure('Name', 'Сравнение влияния скремблирования', 'Color', 'w', 'Position', [100,
100, 1000, 400]);
subplot(1,2,1);
semilogy(EbNo_dB, ber_hard_sc, 'r-o', 'LineWidth', 1.5); hold on;
semilogy(EbNo_dB, ber_hard_no_sc, 'k--s', 'LineWidth', 1.5);
grid on;
title('Жёсткое декодирование (Hard Decision)');
xlabel('Eb/N0 (дБ)');
ylabel('BER');
legend('Со скремблированием', 'Без скремблирования', 'Location', 'northeast');
subplot(1,2,2);
semilogy(EbNo_dB, ber_soft_sc, 'b-o', 'LineWidth', 1.5); hold on;
semilogy(EbNo_dB, ber_soft_no_sc, 'k--s', 'LineWidth', 1.5);
grid on;
title('Мягкое декодирование (Soft Decision)');
xlabel('Eb/N0 (дБ)');
ylabel('BER');
legend('Со скремблированием', 'Без скремблирования', 'Location', 'northeast');
sgtitle('Влияние скремблирования на BER для 64-QAM');
```

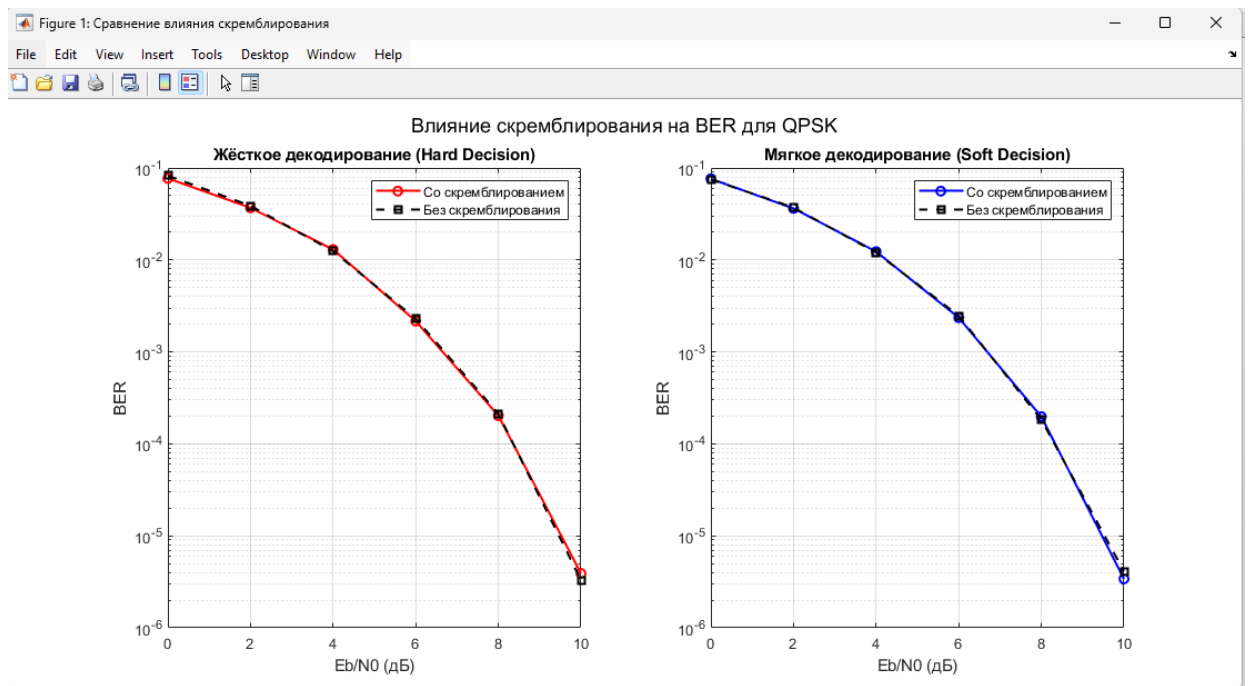


Рисунок 4 - Влияние скремблирования на график BER для QPSK модуляции

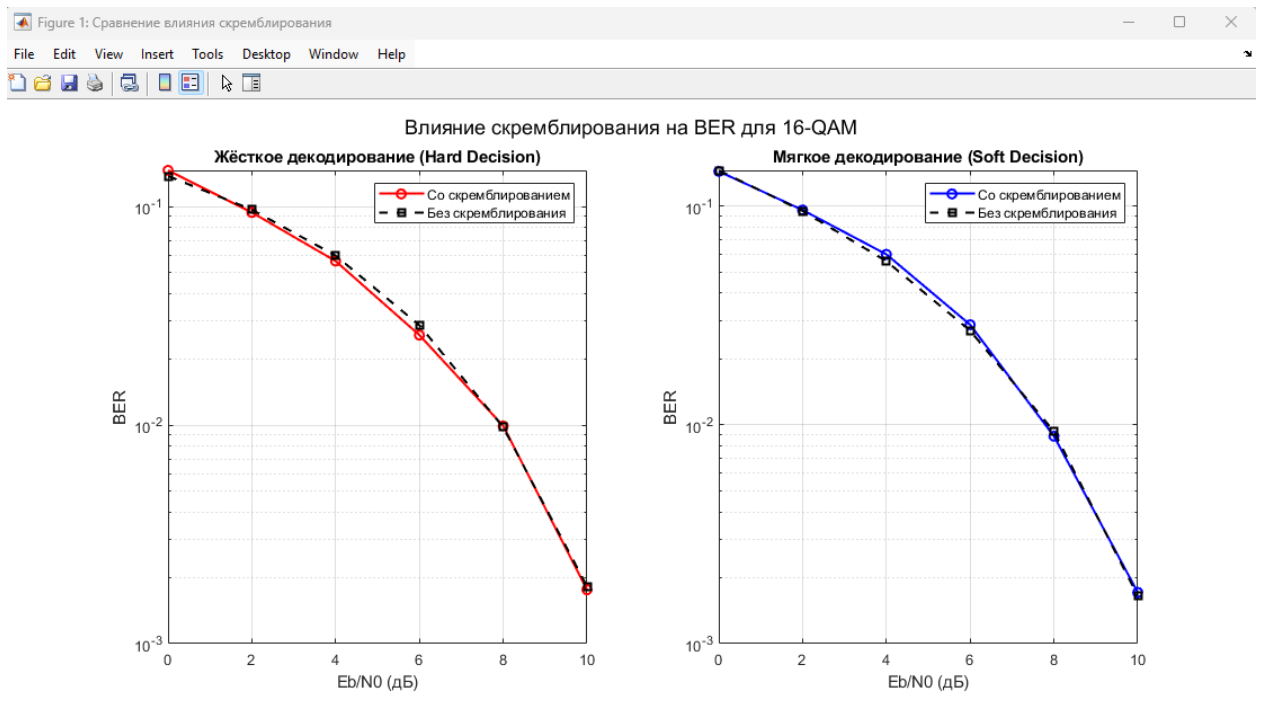


Рисунок 5 - Влияние скремблирования на график BER для 16-QAM модуляции

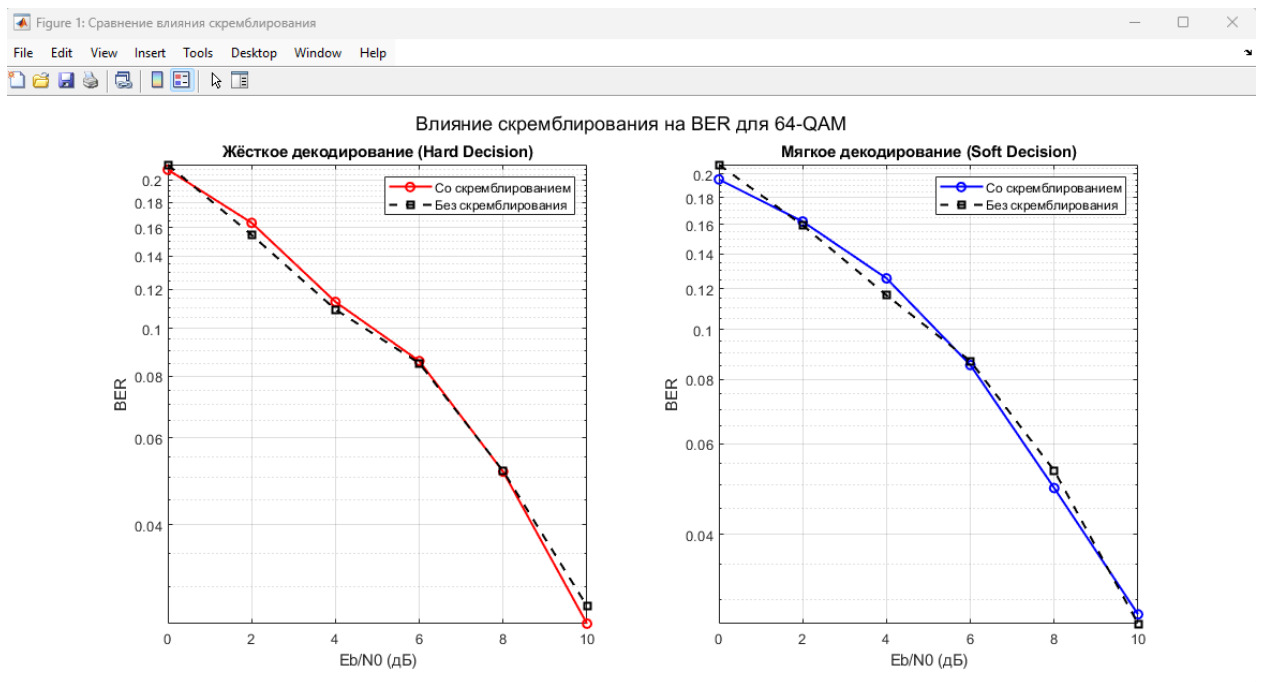


Рисунок 6 - Влияние скремблирования на график BER для 64-QAM модуляции

Скремблирование не изменяет вероятностные характеристики битовых ошибок. Это объясняется тем, что операция исключающего ИЛИ с фиксированной последовательностью является обратимым преобразованием, которое перемешивает биты, но не вносит избыточности и не изменяет расстояние Хэмминга между кодовыми словами. Поскольку в нашей системе отсутствует канальное кодирование, скремблер и дескремблер представляют собой тождественное преобразование для статистики ошибок.

Таким образом, для всех трех типов модуляции QPSK, 16-QAM и 64-QAM наличие или отсутствие скремблирования не приводит к изменению кривых BER в канале АБГШ без канального кодирования, как мы можем заметить на графиках выше. При включенном скремблировании графики будут полностью совпадать с графиками при выключенном скремблировании.

Задание №4: Определение систематической части кодового слова.
Систематический и несистематический код.

Систематическая часть кодового слова представляет собой исходные информационные биты, которые входят в состав кодового слова в неизменном виде. То есть если на вход кодера поступает некоторая последовательность информационных символов, то в выходном кодовом слове эти символы присутствуют явно и занимают определенные позиции, как правило, в начале или в конце. Оставшаяся часть кодового слова содержит проверочные символы, которые вычисляются на основе информационных и служат для обнаружения или исправления ошибок.

Систематический код:

Это код, в котором информационные символы входят в состав кодового слова в явном и неизменном виде. Основное преимущество систематических кодов заключается в удобстве реализации. Приемник может извлечь информационные биты непосредственно из принятого кодового слова, не выполняя операцию декодирования в полном объеме, что упрощает реализацию и снижает задержки. Большинство применяемых на практике кодов, включая коды Хэмминга, циклические коды, коды Рида-Соломона, а также сверточные коды в режиме систематического кодирования.

Несистематический код:

Это код, в котором информационные символы не присутствуют в кодовом слове в явном виде. Входная информационная последовательность преобразуется таким образом, что исходные биты распределяются по всему кодовому слову и не могут быть непосредственно выделены без выполнения полной процедуры декодирования. Хотя несистематические коды могут обладать теми же корректирующими способностями, что и их систематические аналоги, но они реже применяются из-за меньшего удобства извлечения информации. Однако в некоторых случаях, например при использовании турбокодов в определенных режимах, несистематическое представление может давать преимущества эффективности декодирования.

Систематические коды в технологии LTE:

В LTE для высокоскоростных каналов передачи данных, таких как DL-SCH и UL-SCH, используется систематический турбокод со скоростью $1/3$. Структура кодового слова состоит из систематической части, где находится исходная информационная последовательность. Также два потока проверочных символов, формируемых двумя рекурсивными систематическими сверточными кодерами, разделенными перемежителем.

Применение систематического турбокода в LTE:

1. Информационные символы оказываются доступны непосредственно из кодового слова без выполнения полной процедуры декодирования, что упрощает приемник и снижает задержки.
2. Порождающая матрица систематического кода имеет каноническую форму, где единичная матрица соответствует систематической части.
3. Систематическая структура обеспечивает возможность автоматического запроса на повторение с избыточностью, когда в повторных передачах могут отправляться только дополнительные проверочные символы.

Также стоит отметить, что систематичность и несистематичность являются свойствами конкретной реализации кодера, а не самого кода как множества кодовых слов. Любой линейный код может быть представлен как в систематической, так и в несистематической форме путем выбора соответствующей порождающей матрицы. Теоретически корректирующая способность кода не зависит от того, является ли его представление систематическим или несистематическим.

Задание №5: Заполнение таблицы с максимальным кол-вом итераций турбокодера.

В этом задании нам необходимо измерить время выполнения кода турбодекодера в зависимости от максимального количества итераций.

Измерять время мы будем, используя функцию MATLAB **tic** / **toc**. Измерения проводятся для каждого значения максимального количества итераций: 1, 2, 4, 6, 8, 10.

```
% Параметры
blockSize = 1000;
uLength = 3 * blockSize + 12;
iters = [1, 2, 4, 6, 8, 10];

% Подготовка данных
u = randn(uLength, 1);
indices = randperm(blockSize)';
times = zeros(length(iters), 1);

fprintf('Итерации | Время (с)\n');
fprintf('-----\n');

for i = 1:length(iters)
    tic;
    y = TurboDecoder(u, indices, iters(i));
    times(i) = toc;
    fprintf('%8d | %.6f\n', iters(i), times(i));
end
```

```
>> task5
```

Итерации	Время (с)
1	0.057104
2	0.047242
4	0.046910
6	0.044843
8	0.054871
10	0.048205

Рисунок 7 – Таблица для задания №5 (MATLAB)

Таким образом, в проведенном эксперименте все замеры времени соответствуют декодированию с одной итерацией. Небольшие колебания времени от 0.044 до 0.057 секунд объясняются загрузкой процессора и не имеют отношения к количеству итераций.

Начиная со 2 итераций, объект уже инициализирован, и измеряется только время непосредственно декодирования. В диапазоне 2-10 итераций время выполнения составляет 0.044-0.057 секунды, причём увеличение количества итераций с 2 до 10 приводит к росту времени примерно на 30%, что свидетельствует о близкой к линейной зависимости.

В LTE для абонентских устройств часто используется 6-8 итераций как компромисс между качеством декодирования и энергопотреблением, а для базовых станций, где ограничения по энергии менее критичны, может применяться до 12 итераций.

Задание №6: Анализ результатов профилирования. Какие модули приемника занимают наибольшее время на выполнение вычислений?

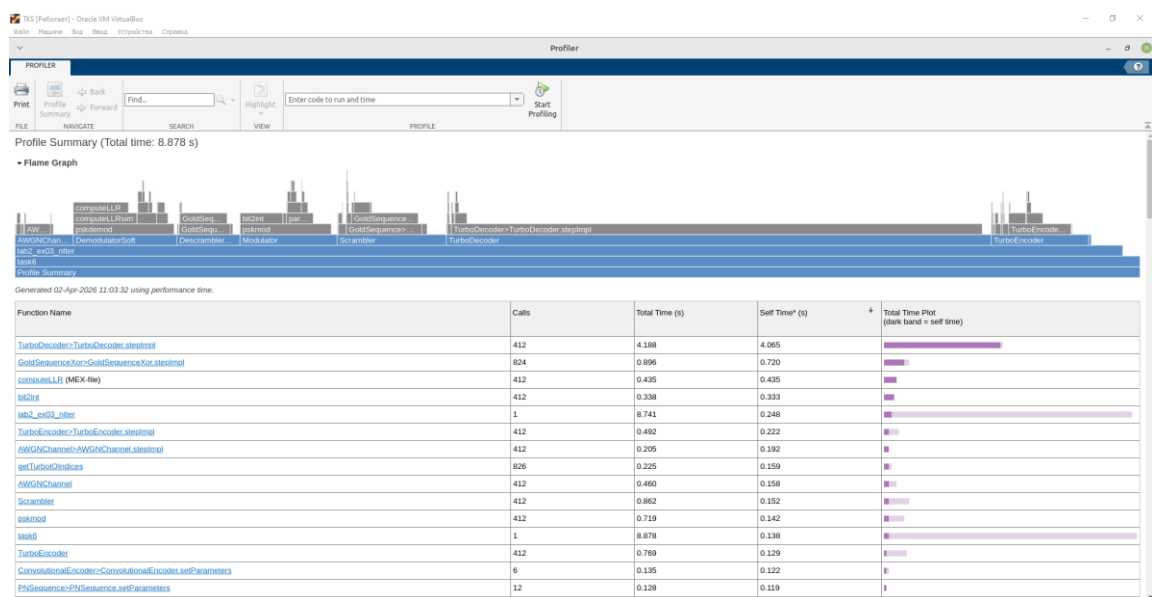


Рисунок 8 – Результаты выполнения кода и анализ профилирования по времени

Самый долгий процесс в коде является работа турбодекодера, которая занимает почти половину всего времени вычислений (4.2 секунды из 8.9 секунд).

Турбодекодер самый сложный и требовательный к ресурсам блок в системе. Его задача заключается в том, чтобы исправлять ошибки, возникшие при передаче сигнала по каналу связи. Для этого он выполняет итеративную процедуру, то есть многократно повторяет одни и те же вычисления, постепенно уточняя результат. Каждая такая итерация включает в себя сложные математические операции такие как: вычисление прямых и обратных рекурсий по алгоритму Макса-Логарифма-МАП, вычисление прямых и обратных рекурсий по алгоритму Макса-Логарифма-МАП и на каждой итерации происходит обмен мягкой информацией между двумя компонентными декодерами. Чем больше итераций требуется, тем дольше работает декодер.

Именно из-за этой итеративной природы и высокой вычислительной сложности турбодекодер становится главным по затратам времени и основным фактором, определяющим общую производительность приемника в стандарте LTE.

Задание №7: Сравнение времени выполнения декодирования с фиксированным кол-вом итерации и с использованием CRC. Сравнение графиков BER для двух случаев. Оформить вывод о целесообразности использования CRC.

```
EbNo = 1;
maxNumErrs = 100;
maxNumBits = 1e5;
tic;
[ber_fix, bits_fix] = lab2_ex04(EbNo, maxNumErrs, maxNumBits);
t_fix = toc;
tic;
[ber_crc, bits_crc] = lab2_ex04_crc(EbNo, maxNumErrs, maxNumBits);
t_crc = toc;
fprintf('Время (фикс. итерации): %.4f c\n', t_fix);
fprintf('Время (с CRC): %.4f c\n', t_crc);
fprintf('Экономия времени: %.1f%%\n', (1 - t_crc/t_fix)*100);
%% Моделирование BER
EbNo_range = 0:0.2:1.2;
ber_fixed = zeros(size(EbNo_range));
ber_early = zeros(size(EbNo_range));
for i = 1:length(EbNo_range)
    en = EbNo_range(i);
    [ber_fixed(i), ~] = lab2_ex04(en, maxNumErrs, maxNumBits);
    [ber_early(i), ~] = lab2_ex04_crc(en, maxNumErrs, maxNumBits);
    fprintf('Eb/No = %.1f dB: ber_fixed = %.2e, ber_early = %.2e\n', en, ber_fixed(i), ber_early(i));
end
figure('Position', [100, 100, 800, 600]);
semilogy(EbNo_range, ber_fixed, '-bo', 'LineWidth', 1.5, 'MarkerSize', 8);
hold on;
semilogy(EbNo_range, ber_early, '--rx', 'LineWidth', 1.5, 'MarkerSize', 8);
grid on;
xlabel('E_b/N_0 (дБ)');
ylabel('Bit Error Rate (BER)');
```

```

title('Сравнение BER: фиксированные итерации vs досрочный выход (CRC)');
legend('Фиксированные итерации (6)', 'Досрочный выход (CRC)', 'Location',
'southwest');
ylim([1e-5, 1]);

```

Результат выполнения кода:

Время (фикс. итерации): 3.6351 с

Время (с CRC): 2.3235 с

Экономия времени: 36.1%

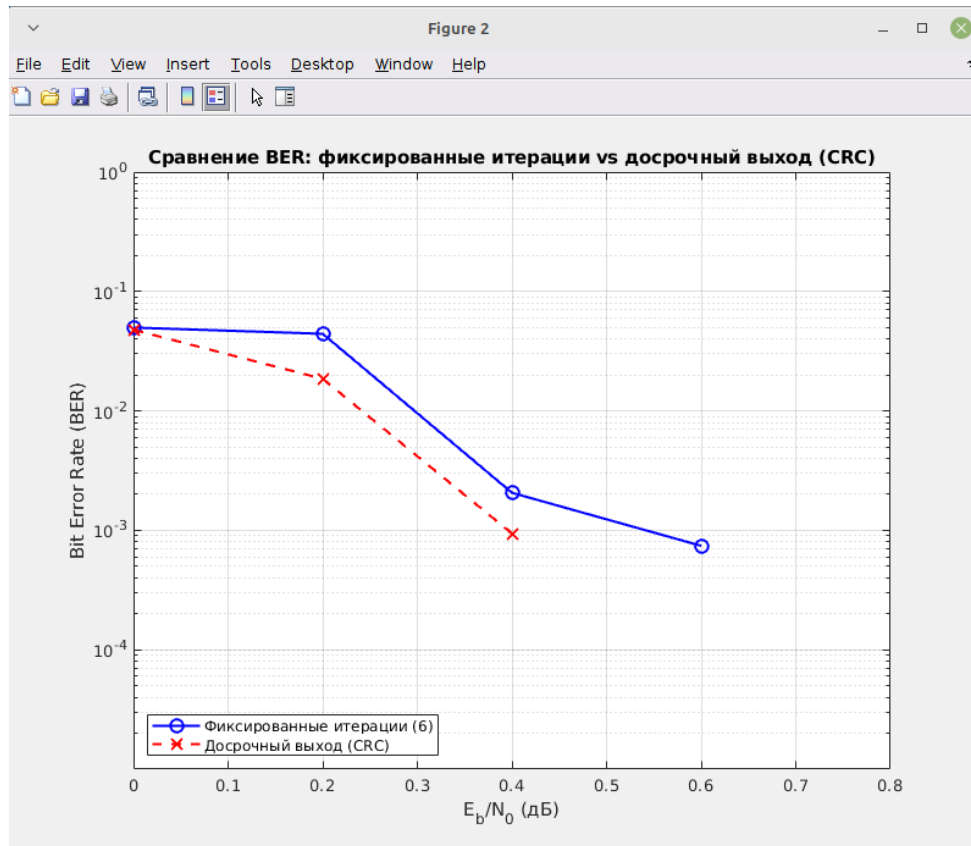


Рисунок 9 - Сравнение BER для фиксированных итераций и досрочного выхода

Сравнение времени выполнения показывает, что использование механизма досрочного выхода с CRC позволяет сократить время декодирования на 36.1% при $E_b/N_0 = 1$ дБ. Это объясняется тем, что при достаточно высоком отношении сигнал/шум декодер успешно восстанавливает данные за меньшее количество итераций, чем заданное максимальное значение (6 итераций), и досрочно завершает работу после проверки CRC.

Сравнение графиков BER демонстрирует, что помехоустойчивость при использовании досрочного выхода практически не ухудшается по сравнению с фиксированным количеством итераций. Незначительные различия находятся в пределах статистической погрешности моделирования.

Таким образом, применение механизма досрочного выхода из цикла декодирования с использованием CRC является целесообразным, так как позволяет снизить вычислительную сложность (время обработки) без заметного ухудшения

помехоустойчивости системы. Это особенно важно для систем реального времени, где требуется минимизация задержек обработки сигнала и энергопотребления.